

Students Cluster User Guide

Paolo Joseph Baioni

April 6, 2026

Contents

Essential usage and cheatsheet	1
1 Access	3
2 General Structure	3
2.1 File Systems	4
3 Usage	4
3.1 Job Scheduler	5
3.1.1 Queues	5
3.2 Software	5
3.2.1 Additional software examples (MATLAB, PETSc, deal.II)	5
3.2.2 Local software	7
3.2.3 Spack	7
3.2.4 Python, environments, Jupyter and Matplotlib	7
3.2.5 Apptainer	8
3.3 Examples	8
3.3.1 Interactive jobs	8
3.3.2 Batch jobs	8
3.3.3 PBS + Apptainer	9
3.3.4 PBS + Spack + MPI	11
3.3.5 GPUs and Apptainer	11
3.4 Recommended workflow	12
4 Storage policy and recommended usage	12
4.1 Storage overview	12
4.2 Important note on reliability	12
4.3 Recommended workflow	12

Essential usage and cheatsheet

This section contains the minimal information needed to use the cluster. For more detail, see the following sections.

Minimal workflow

1. Connect

- Register for the course and follow the VPN guide on WeBeep.
- Activate the VPN (`gp-dmat-saml.vpn.polimi.it`).
- Log in (replace `username`):

```
ssh username@10.78.18.100
```

2. **Enable PBS commands** (first login only or in new shells):

```
. /etc/profile.d/pbs.sh
```

3. **Run a short interactive CPU job with default resources** (default queue `cpu`, inspect with `qstat -Qf cpu`):

```
qsub -I
```

Wait until the prompt shows a compute node (e.g. `cpu01`), run your commands, then:

```
exit
```

4. **Run a short interactive GPU job** (1 GPU, 2 threads):

```
qsub -I -q gpu -l select=1:ncpus=2:ngpus=1 -l walltime=00:10:00
```

5. **Submit a batch job script** (CPU example):

```
cat > cpu_job.sh << 'EOF'
#!/bin/bash
#PBS -N hello_cpu
#PBS -q cpu
#PBS -l select=1:ncpus=2
#PBS -l walltime=00:05:00
#PBS -j oe

echo "Running on: $(hostname)"
python -c "print('Hello from CPU job')"
EOF

qsub cpu_job.sh
```

Monitoring and control

- List all jobs: `qstat`
- List only your jobs: `qstat -u $(whoami)`
- Detailed information on a job: `qstat -f <JOBID>`
- Cancel a job: `qdel <JOBID>`
- Show queues: `qstat -Q`
- Show nodes: `pbsnodes -aSj`

Remark It is possible to use Linux commands for continuous monitoring; e.g. `watch qstat -u $(whoami)` performs `qstat` every 2.0 s.

Filesystem usage (recommended pattern)

- **Home** `/home/username`
 - Code, configuration files, small data, virtual environments, small Spack trees.
 - Smaller quota, RAID 1.

- **Work** `/work/username`
 - Project data and results that you care about.
 - Medium quota, RAID 5 (more resilient, lower throughput than previous configuration).
- **Global scratch** `/scratch_global`
 - Large, temporary data; no quotas; auto-clean after 30 days.
- **Local scratch on nodes** `/scratch_local`
 - Fast, node-local temporary storage (8.7 TB on CPU nodes, 10.5 TB on GPU nodes).
 - Files older than 30 days are deleted automatically on each node.
 - Use inside jobs for I/O-intensive work; copy important data back to `/work` or `/home` before the job ends.

Important rules

- Do *not* run computations on `login01`. The login node is only for access, job submission, file transfer.
- Any user process whose average CPU utilisation on the login node exceeds 20% of one logical core will be terminated automatically.
- Always run production work via PBS (interactive with `qsub -I` or batch with a job script).

1 Access

Register for the course at <https://webeep.polimi.it/course/view.php?id=14384>. In *Materials*, users will find the VPN guide and the *Students* directory, which contains this guide and a link to the Microsoft Forms page for account requests.

Accounts are created on Mondays and Wednesdays and are available from the following day. All usernames are of the form `upersoncode`, for example `u12345678`.

To log in:

- activate the VPN (`gp-dmat-saml.vpn.polimi.it`)¹
- run `ssh username@10.78.18.100`

If users have multiple keys, or a key in a non-standard location / with a non-standard name, defaults can be overridden with:

```
1 ssh username@10.78.18.100 -i ~/.ssh/id_ed25519
```

In case of problems, the connection can be inspected with `ssh -vv`.

2 General Structure

Once logged in, users will be in their home directory at `/home/username` on the login node, named `login01`. The prompt should display something like:

```
username@login01:~$
```

In the cluster there are:

- the login node, which, from the user perspective, serves only as a login and job submission node (see the next sections). Any job using more than 15% of a single CPU thread will be automatically killed;

¹See VPN guide on WeBeep: *Materials* → *Instructions for first access*.

- general compute nodes, divided into 5 CPU nodes (112 hardware threads, 0.5 TB RAM, 8.7 TB local scratch) and 2 multi-GPU nodes (112 hardware threads, 1 TB RAM, 10.5 TB local scratch, and 2 NVIDIA L4 GPUs each);
- specific project nodes, with dedicated queues whose access might be granted in special cases (e.g. thesis work on the topic of the project), subject to approval. Precise configurations may vary depending on project needs and they will not be considered further in this document; however, the general considerations here apply to them as well.

In addition, internal maintenance queues may exist; access to these is restricted and managed by the system administrator.

2.1 File Systems

The default permissions for `/home/username` are 0700 and the default owner is `username:username`. Users also have a directory at `/work/username`; default permissions are 0750 and the default owner is again `username:username`.²

`/home` and `/work` are file systems with quota enabled, which means that users can occupy up to a certain amount of space. This is set to approximately 10 GB and 50 GB for home and work respectively (soft limits; hard limits slightly higher). Home is configured in RAID 1, work in RAID 5.

Users also have write access (permissions 1777) to a shared, no-quota `/scratch_global` file system (RAID 0) and to a similar `/scratch_local`, *local* to each node. `/scratch_global` and `/scratch_local` are temporary file systems; files older than 30 days will be deleted automatically.

Finally, users have read and execute access to `/software`, which contains useful libraries and container images.

`/home`, `/work`, `/software` and `/scratch_global` are shared across the cluster via NFSv4 over 1 Gbps Ethernet. This means that:

- they will be available on any node users have access to;
- their performance is comparatively slow, with the network being the bottleneck.

On each compute node there is also local scratch: 8.7 TB on CPU nodes and 10.5 TB on GPU nodes, both in RAID 0 and both temporary file systems (30-day retention). As they are local to each node, they are significantly faster than the global file systems and should be used for I/O-intensive workloads. Data can then be copied to personal directories or global scratch and transferred to personal devices from the login node with `scp`, e.g.

```
scp -r username@10.78.18.100:/home/$(whoami)/dir .
```

(see also `man scp` on the cluster).

GPU nodes use NVMe SSDs, while CPU and login nodes use HDDs; thus, local scratch on GPU nodes is faster.

3 Usage

As anticipated, the login node can be used only for:

- access (`ssh`);
- job submission, jobs, queues and nodes monitoring (PBS);
- file transfer (e.g. `scp`).

Any user process whose average CPU utilisation on the login node exceeds 20% of one logical core will be terminated automatically.

²Users can change these at their own risk with `chmod` and `chown` respectively.

3.1 Job Scheduler

For job submission, the cluster uses OpenPBS; the source code can be inspected in `/software/openpbs`. Although there is not always a one-to-one correspondence, interested readers can refer to the Altair PBS Pro documentation, for example: [PBS Professional 2024.1 manual](#). Basic instructions and examples follow.

Note that at first login the PBS binaries are not in the `PATH`. Users are free to use the full path, or source a system script:

```
1 u12345678@login01:~$ which pbsnodes # no output means not found
2 u12345678@login01:~$ . /etc/profile.d/pbs.sh # source shell script
3 u12345678@login01:~$ which pbsnodes
4 /opt/pbs/bin/pbsnodes # /opt/pbs/bin/ is now in the PATH
```

In what follows, it is assumed that users have sourced the script; if not, prepend `/opt/pbs/bin/` to PBS commands such as `qsub`, `qstat`, `qdel` and `pbsnodes`.

3.1.1 Queues

Existing queues can be inspected with `qstat -Q`, and nodes with `pbsnodes -aSj`. In particular, two queues are available to all users:

- `cpu`, which handles the 5 CPU nodes;
- `gpu`, which handles the 2 multi-GPU nodes.

Both queues have limits on the maximum resources that can be requested and on the maximum number of jobs that can be submitted. These can be inspected with:

```
qstat -Qf <queue_name>
```

If no queue is specified, the default queue (`cpu`) will be used.

3.2 Software

Each node has both locally installed software and software shared via NFS, the latter under `/software`. The remainder of this section describes tools and workflows that are useful but not strictly required for basic use of the cluster.

3.2.1 Additional software examples (MATLAB, PETSc, deal.II)

MATLAB MATLAB is available on the cluster via system installation and can be executed in batch mode. In general, it is sufficient to invoke the executable directly using its full path, for example:

```
/software/MATLAB/R2025b/bin/matlab -batch "run('script.m')" -singleCompThread
```

The `-batch` flag runs MATLAB non-interactively, while `-singleCompThread` ensures that only one CPU thread is used for serial jobs. When writing PBS scripts, it is recommended to first test commands interactively (`qsub -I`) and then replicate the working sequence in batch mode.

PETSc PETSc (version 3.19.6) is available on the cluster and can be used for parallel numerical computations. A simple check of the installation can be performed with:

```
pkg-config --modversion PETSc
```

Example compilation and execution:

```
qsub -I
nano ~/hello_petsc.cpp
mpicxx ~/hello_petsc.cpp -o ~/hello_petsc $(pkg-config --cflags --libs PETSc)
ctrl+D
```

```
qsub -I -q cpu -l select=1:ncpus=4:mpiprocs=4
mpirun -n 4 ~/hello_petsc
ctrl+D
```

Minimal example:

```
#include <petscsys.h>
#include <iostream>

int main(int argc, char **argv) {
    PetscInitialize(&argc, &argv, nullptr, nullptr);
    int rank;
    MPI_Comm_rank(PETSC_COMM_WORLD, &rank);
    std::cout << "Hello from PETSc! Rank: " << rank << std::endl;
    PetscFinalize();
    return 0;
}
```

deal.II The finite element library deal.II (version 9.7.1) is installed on the system. When compiling external examples, it may be necessary to adjust the required version in the CMake configuration:

```
find_package(deal.II 9.7.1
    HINTS ${deal.II_DIR} ${DEAL_IIDIR} ../ ../.. $ENV{DEAL_IIDIR}
)
```

Example CMakeLists.txt:

```
cmake_minimum_required(VERSION 3.13.4)

find_package(deal.II 9.7.1
    HINTS ${deal.II_DIR} ${DEAL_IIDIR} ../ ../.. $ENV{DEAL_IIDIR}
)

deal_ii_initialize_cached_variables()

add_executable(hello_dealii hello_dealii.cpp)
deal_ii_setup_target(hello_dealii)
```

Example minimal program:

```
#include <deal.II/base/logstream.h>
#include <iostream>

using namespace dealii;

int main()
{
    deallog.depth_console(0);
    std::cout << "Hello from deal.II!" << std::endl;
    return 0;
}
```

Compilation:

```
qsub -I
mkdir ~/dealii_example
cd ~/dealii_example
nano CMakeLists.txt
nano hello_dealii.cpp
mkdir build
cd build
cmake ..
make
./hello_dealii
```

Additional configuration files are available under `/usr/share/cmake/deal.II`.

3.2.2 Local software

Besides essential software (`gcc`, `git`, `vim`, `nano`, `make`, `python`, to name a few), Apptainer, GNU Octave and FreeFem++ are installed locally and can therefore be launched directly; the latter is installed only on `cpu` nodes.

3.2.3 Spack

Various Spack³ installations are available under `/software`; a link to the default one is `/software/spack`. For example, to compile and run a C++ program with a recent compiler, relying on TBB and Open MPI for parallelisation:

```
1 source /software/spack/share/spack/setup-env.sh
2 spack load gcc@15.2.0
3 spack load intel-oneapi-tbb@2022.3.0
4 spack load openmpi@5.0.8
```

Successful loading can be checked with, for example, `gcc --version` or `which gcc`. Available Spack packages can be listed with `spack find` and unloaded with `spack unload --all`.

Users can also build their own toolchains, for example:

```
1 git clone -c feature.manyFiles=true --depth=1 --branch releases/v1.0 https://github.com/
   spack/spack.git ~/spack-v1.0
2 source ~/spack-v1.0/share/spack/setup-env.sh
3 spack list
4 spack info gcc
5 spack install gcc@15.2.0
6 spack load gcc@15.2.0
7 spack compilers find
8 spack install intel-oneapi-tbb@2022.3.0%gcc@15.2.0
```

3.2.4 Python, environments, Jupyter and Matplotlib

Users can create virtual environments in their home directories, or use any environment already available. For example, it is possible to use the environment `/software/pyplot` to produce Matplotlib plots via the Jupyter CLI:

```
1 source /software/pyplot/bin/activate
2 nbterm notebook.ipynb
3 deactivate
```

(see <https://github.com/davidbrochart/nbterm> for `nbterm` commands).

For plots, note that images should be saved and then copied with `scp` to a local device for viewing. As an example of environment creation, such an environment may be created with:

```
1 python -m venv ~/my_env
2 source ~/my_env/bin/activate
3 pip install --upgrade pip
4 pip install numpy matplotlib jupyter nbterm
```

If a specific Python version is required or useful, different from the system one, it is possible to use Spack to install and load it.

³<https://github.com/spack/spack>

3.2.5 Apptainer

To install specific software that might be missing on the cluster, especially when portability and reproducibility are important, one option is to use [Apptainer](#), an open-source container platform designed to run on HPC systems.⁴

Apptainer is already installed locally on each node; as with Spack, users can install their own version following the Apptainer documentation if needed. For example:

```
1 mkdir -p ~/opt/apptainer
2 curl -s https://raw.githubusercontent.com/apptainer/apptainer/main/tools/install-
  unprivileged.sh | bash -s - ~/opt/apptainer
3 ~/opt/apptainer/bin/apptainer --help
```

Containers can be built from Docker or Podman images—for example from [Docker Hub](#), [Quay.io](#) and [NVIDIA NGC](#)—or from text files, called *recipes*; see, for example, <https://github.com/HPC-Courses/AMSC-Labs/>, in particular:

- [2025-26/00-environment_setup/README.md](#)
- [2025-26/01-scientific-computing-tools/hpsc_tools.md #4-containerisation-in-hpc](#)

Execution of any container can be performed both interactively (via `qsub -I`) and in batch mode. In the case of large container images, it may be necessary to use ad hoc temporary and cache directories, e.g.

```
1 mkdir -p ~/apptainer_tmp
2 export APPTAINER_TMPDIR=~/apptainer_tmp
3 export APPTAINER_CACHEDIR=~/apptainer_tmp
```

Support for GPUs can be enabled when running with `--nv` (NVIDIA); this is particularly important for containers pulled from <https://catalog.ngc.nvidia.com/>, such as the one already available at [/software/containers/nvhpc](#).

3.3 Examples

3.3.1 Interactive jobs

To request an interactive job with default resources, run:

```
1 qsub -I
```

and, when finished:

```
1 logout
```

It is also possible to specify particular resources for specific needs, for example:

```
1 qsub -I -q gpu -l select=1:ncpus=1:ngpus=1
```

which requests interactive access to the `gpu` queue, asking for 1 CPU thread and 1 GPU on a single node.⁵ This mode is particularly useful for short jobs and for development and testing.

3.3.2 Batch jobs

Once testing is complete, for longer jobs it is advisable to use batch mode. Batch mode consists of specifying the resources required and the commands to be executed in a script, and then submitting that script to PBS.

For example:

⁴See <https://apptainer.org/docs/user/latest/> and <https://github.com/apptainer/apptainer>.

⁵GPUs are always shared.

```

1 #!/bin/bash
2 #PBS -l select=5
3 #PBS -l place=scatter
4 #PBS -l walltime=00:05:00
5 #PBS -q cpu
6 #PBS -N hello_scatter
7
8 # Print each node name
9 for node in $(uniq "$PBS_NODEFILE"); do
10     echo "hello from pbs on node $node" \
11         >> "$HOME/hello.scatter.$PBS_JOBID"
12 done

```

This will produce a file named `hello.scatter.<JOBID>.login01`, containing, for example:

```

hello from pbs on node cpu01
hello from pbs on node cpu02
hello from pbs on node cpu03
hello from pbs on node cpu04
hello from pbs on node cpu05

```

3.3.3 PBS + Apptainer

Docker Hub & interactive, system version In the simplest case, it is enough to ask for an interactive session:

```

1 qsub -I

```

then build the container from Docker Hub:

```

1 apptainer pull ~/sagemath.sif docker://sagemath/sagemath

```

and run it:

```

1 apptainer run ~/sagemath.sif

```

which launches SageMath, for example:

```

SageMath version 10.4, Release Date: 2024-07-19
Using Python 3.12.4. Type "help()" for help.
sage:

```

Use `Ctrl+D` to exit.

Recipe & interactive, user version When the software is not present on Docker Hub or other hosting services, the container can be built starting from a user-defined recipe. In order to do so, it is possible to start from a Linux distribution which is known to the user and where the user would be able to install the software using the standard package manager.

Here we show how to build Macaulay2 (<https://www.macaulay2.com/>) with Ubuntu commands, using the base OS from Docker Hub.

The first step is to write a text file, here called `m2_minimal.def`. Its content can be:

```

1 BootStrap: docker
2 From: ubuntu:22.04
3 %post
4     apt-get update -y
5     DEBIAN_FRONTEND=noninteractive apt-get install -y --no-install-recommends \
6         macaulay2
7     rm -rf /var/lib/apt/lists/*
8 %runscript
9 M2

```

As before, we can use either the system version of Apptainer or a user-installed one. In the second case we first build the container with:

```
1 qsub -I
2 ~/opt/apptainer/bin/apptainer build m2_minimal.sif m2_minimal.def
```

and then run it with:

```
1 ~/opt/apptainer/bin/apptainer run m2_minimal.sif
```

which gives, for example:

```
Macaulay2, version 1.19.1
with packages: ConwayPolynomials, Elimination, IntegralClosure, InverseSystems,
LLLBases, MinimalPrimes, PrimaryDecomposition,
ReesAlgebra, Saturation, TangentCone
i1 : help
o1 = initial help
*****
Welcome to Macaulay2
```

It is also possible to use Spack inside the recipe, for example cloning Spack after installing essential packages with apt.

Recipe & batch job, system version It is also possible to submit batch jobs to PBS which rely on containers. In this example we build a container for the GNU Ada compiler and use it to compile and run an Ada “hello world” via PBS.

First, we write the recipe, using e.g. Rocky Linux as base OS:

```
1 BootStrap: docker
2 From: rockylinux:9.3
3 %post
4 dnf install -y --setopt=install_weak_deps=False epel-release
5 dnf install -y --setopt=install_weak_deps=False gcc-gnat
6 rm -rf /var/cache/dnf/*
```

We call this recipe gnat.def and build the container with:

```
1 apptainer build gnat.sif gnat.def
```

We then write an Ada hello world for testing, called hello.adb:

```
with Ada.Text_IO; use Ada.Text_IO;
procedure Hello is
begin
  Put_Line ("Hello WORLD!");
end Hello;
```

Next we write a PBS job submission script, gnat.pbs:

```
1 #!/bin/bash
2 #PBS -l select=1:ncpus=1
3 #PBS -l walltime=00:05:00
4 #PBS -N gnat
5
6 apptainer exec gnat.sif gnatmake hello.adb
7 apptainer exec gnat.sif gnatbind -x hello.ali
8 apptainer exec gnat.sif gnatlink hello.ali
9 apptainer exec gnat.sif ./hello
```

Finally, we submit the job with:

```
1 qsub gnat.pbs
```

Messages directed to standard error can be inspected by looking into the `<jobname>.e<jobid>` file, in this case:

```
1 cat gnat.e3221
2 x86_64-linux-gnu-gcc-11 -c hello.adb
3 x86_64-linux-gnu-gnatbind-11 -x hello.ali
4 x86_64-linux-gnu-gnatlink-11 hello.ali
```

Messages directed to standard output can be inspected in the `<jobname>.o<jobid>` file, in this case:

```
1 cat gnat.o3221
2 Hello WORLD!
```

3.3.4 PBS + Spack + MPI

This example uses PBS interactively; the same specifications can be applied in batch mode. In particular:

- the requested resources correspond to 2 MPI processes, each of which requires 3 CPU threads, resulting in `ncpus=6`;
- the subsequent commands can be placed after the last `#PBS -l` line in the submission script, possibly specifying an output and/or error file, as in the previous examples.

It is possible to obtain the source code from: <https://github.com/HPC-Courses/AMSC-Labs/tree/main/Labs/2025-26/08-mpi-stl>

```
1 qsub -I -q cpu -l select=1:ncpus=6:mpiprocs=2:host=cpu05
2 source /software/spack/share/spack/setup-env.sh
3 spack load gcc@15.2.0
4 spack load intel-oneapi-tbb@2022.3.0
5 spack load openmpi@5.0.8
6 mpicxx -std=c++23 -Ofast -march=native -o solution solution.cpp -ltbb
7 mpirun -np 2 ./solution 256 256 16000
```

3.3.5 GPUs and Apptainer

Example of SAXPY on GPU vs CPU with NVIDIA NVHPC:

```
1 qsub -I -q gpu
2 apptainer shell --nv /software/containers/nvhpc.sif
3 nvcc /software/containers/nvhpc/saxpy.cu -DNDEBUG -O4 -o \
4     saxpy.cu -DTHRUST_DEVICE_SYSTEM=THRUST_DEVICE_SYSTEM_CPP
5 nvcc /software/containers/nvhpc/saxpy.cu -arch=sm_89 -DNDEBUG -O4 \
6     -o saxpy.gpu
7 ./saxpy.cu
8 ./saxpy.gpu
```

Note that this provides a meaningful comparison of GPU-parallel versus CPU-*serial* performance only because `THRUST_DEVICE_SYSTEM=THRUST_DEVICE_SYSTEM_CPP` is enforced. Even specifying `ncpus=1` would not guarantee a purely serial run: when `ncpus=1` is set, at least one logical thread is reserved and only one appears as allocated in `pbsnodes -aSj`, but if additional threads are available they may still be used, provided the code can exploit multiple threads.

Therefore, for scaling studies a strict limit on resources employed should be enforced in the code itself.

3.4 Recommended workflow

For a complete example of a typical HPC workflow on this cluster (interactive job, software setup, execution, and data handling), users are encouraged to refer to:

- [pacs-Labs: Remote development environment and cluster workflow](#)

This material provides a step-by-step guide covering environment setup, PBS usage, software management, and a full execution example using MPI and Jupyter-based workflows.

4 Storage policy and recommended usage

4.1 Storage overview

The cluster provides multiple file systems with different performance and reliability characteristics:

- **Local scratch** `/scratch_local` This is the correct location for all I/O-intensive workloads (both single-node and MPI jobs). It is fast, node-local, and temporary.
- **Global scratch** `/scratch_global` Intended as a temporary transfer buffer for large files. It should not be used for heavy I/O workloads or long-term storage.
- **Home** `/home` Limited quota (10 GB). Use for source code, scripts, configuration files, and small final results.
- **Work** `/work` Persistent working area (approximately 50 GB per user). Suitable for software environments and toolchains (e.g. Spack builds, Apptainer images, Python environments). Not intended for heavy I/O workloads.

4.2 Important note on reliability

The `/work` filesystem is configured for a balance between capacity and redundancy, but it must not be considered a backup solution. Hardware failures may still lead to data loss.

No filesystem on the cluster should be considered a reliable backup. Users are responsible for maintaining copies of important data on external systems.

4.3 Recommended workflow

A safe and efficient workflow is:

1. Perform I/O-intensive computations in `/scratch_local`;
2. Verify the results;
3. Copy relevant data to `/scratch_global`, `/work` or `/home`;
4. Transfer important data to your personal machine (e.g. using `scp`);
5. Use version control systems (e.g. GitHub, GitLab) for source code.

Following this workflow ensures both good performance and data safety.